ДИСЦИПЛИНА **Математическая логика и теория алгоритмов**
(полное наименование дисциплины без сокращений)

ИНСТИТУТ **информационных технологий**

КАФЕДРА **прикладной математики**
(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Лекция**
(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **Батенков Кирилл Александрович**
(фамилия, имя, отчество)

СЕМЕСТР **1, 2024-2025**
(указать семестр обучения, учебный год)

1. ЛИНЕЙНЫЕ АЛГОРИТМЫ

1.1. Линейные алгоритмы, их свойства, сферы применения

Линейный алгоритм представляет собой последовательность шагов (действий) без ветвлений и возвратов [28]. Последовательность исполнения шагов такого алгоритма полностью совпадает с его структурой.

Линейный алгоритм представляет собой простую последовательность шагов, которые исполняются в том порядке, в котором они перечислены в алгоритме. В линейном алгоритме не может быть ветвлений и возвратов.

Рассмотрим несколько примеров задач на составление линейных алгоритмов.

Составить блок-схему алгоритма, решающего следующую задачу. Даны три вещественных положительных числа a, b и c. Найти площадь треугольника, стороны которого равны a, b и c.

Для нахождения площади треугольника по трем его известным сторонам воспользуемся формулой Герона:

$$S = \sqrt{p(p-a)(p-b)(p-c)},$$

где p – полупериметр треугольника, равный

$$p = \frac{a+b+c}{2}.$$

Зная последовательность вычислений легко составить алгоритм (рисунок 33).

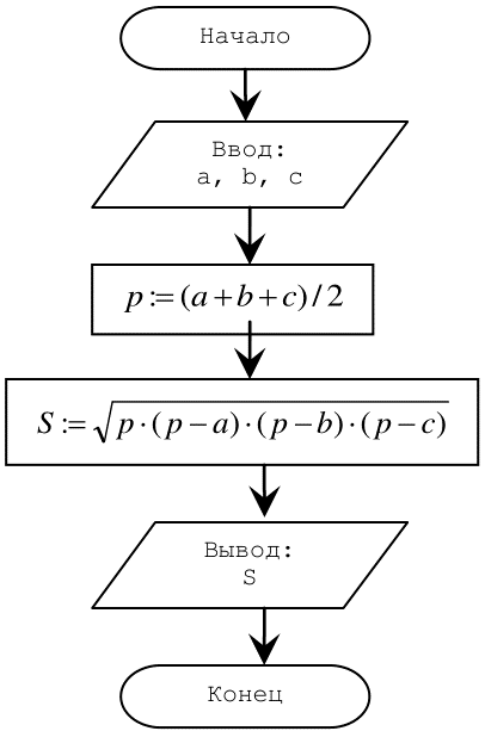


Рисунок 1. Блок-схема алгоритма нахождения площади треугольника

Задача решена. Сделаем два замечания. Первое: в отличие от математики, где приводимые формулы декларируют некоторые факты, соотношения и порядок

указания которых не так важен (или важен только с точки зрения логичности изложения), в алгоритмических схемах важна в первую очередь правильная последовательность действий (формул), которая и определяет порядок выполнения шагов. Например, если в приведенной блок-схеме переставить местами шаги, вычисляющие S и p, то алгоритм не будет правильным, так как до вычисления S необходимо предварительно вычислить p.

Второе замечание. В решении этой задачи никак не рассматривается вопрос существования треугольника, площадь которого вычисляется. То есть мы предполагаем, что входные данные должны быть корректны. В данном случае должны выполняться условия существования треугольника: $a < b + c, b < a + c, c < a + b$. Алгоритм не может быть успешно исполненным, если эти неравенства не выполняются.

Составить блок-схему решения следующей задачи. Даны значения двух действительных переменных a и b. Обменять местами их значения, то есть добиться, чтобы a получила значение, которое изначально имела переменная b, а b – получила бы значение a.

Если первым же присваиванием алгоритма мы переменной a присвоим b, то сразу же потеряем исходное значение a. Поэтому воспользуемся для временного хранения исходного значения переменной a дополнительной переменной d. Блок-схема алгоритма приведена ниже (рисунок 34).

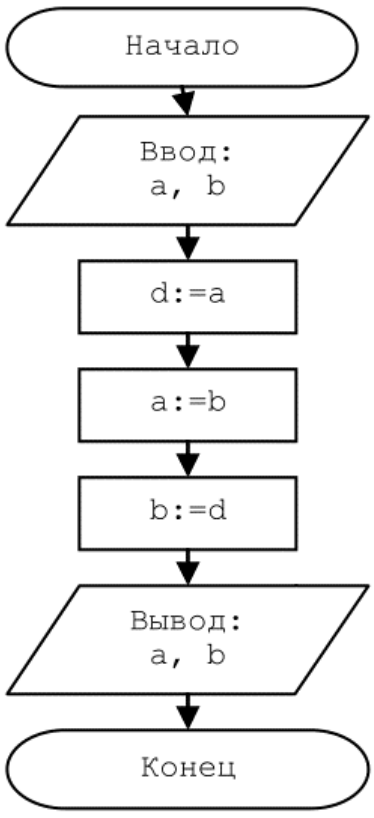


Рисунок 2. Блок-схема алгоритма обмена значений

Составить блок-схему решения следующей задачи. Даны значения двух действительных переменных Рисунок 7. Решение задачи без дополнительных переменных a и b. Обменять местами их значения без использования дополнительных переменных.

В предыдущем примере решалась та же задача, но сейчас запрещается использовать какие-либо переменные, кроме самих a и b. Казалось бы, это невозможно, однако, можно найти и такое решение, причем не одно! Блок-схема решения – на рисунке 35.

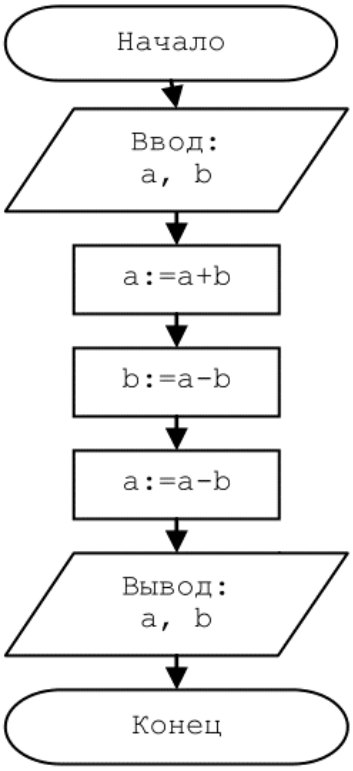


Рисунок 3. Решение задачи без дополнительных переменных

Простыми можно считать такие задачи, у которых алгоритм решения линейный, т. е. все действия выполняются в порядке записи описывающих их операторов [29]. Простые задачи сводятся к линейному процессу со следующими основными пунктами:

ввести исходные данные;

выполнить вычисления по формулам;

вывести результат.

Последовательность решения должна складываться из следующих этапов.

1. Анализ условия задачи.

2. Определение необходимых входных данных: сколько их, какого типа каждое данное, какое имя следует присвоить переменной, обозначающей данное. Тип переменной должен быть определен в объявлении, а имя должно быть значимое, т. е. подчеркивать назначение переменной, ее логический смысл.

3. Определение выходных данных программы: сколько их, какие типы, как назвать переменные для обозначения результатов.

4. Определение списка рабочих переменных, если необходимо.

5. Определение необходимых формул вычисления, их последовательности и правил записи.

6. Кодирование алгоритма.

При кодировании необходимо соблюдать синтаксические правила и следить за стилем программирования. Эти требования можно сформулировать следующим образом.

1. Объявить все переменные программы.

2. Правильно оформить ввод данных. Перед каждым оператором ввода необходимо записать инструкцию, которая подскажет пользователю, что он должен сделать в данный момент времени.

3. Правильно записать формулы вычислений.

4. Грамотно оформить вывод результата. Вывод следует дополнить необходимыми заголовками и пояснениями, числовые данные форматировать.

Пусть требуется вычислить длину окружности, площадь круга и объем шара одного и того же радиуса.

Опишем последовательность решения.

1. Анализ алгоритма показывает, что он линейный.

2. Входная переменная одна, это значение радиуса, в общем случае величина вещественная. Обозначим его именем r.

3. Выходных данных три. Обозначим их именами I, s, v. Все переменные имеют вещественный тип.

4. Формулы вычислений общеизвестны.

```
#include <stdio.h>
// библиотека стандартного ввода-вывода
#include <math.h>
// библиотека математических функций
#define PI 3.1415926 //значение константы π
void main(void)
{
    float r;        // объявление всех переменных
    float I, s, v;
    printf("\nДлина окружности, площадь круга и объем шара :\n");
    // вывод заголовка
    printf("\nВыведите значение радиуса:\n");
    // приглашение для ввода данных
    scanf("%f", &r);    // ввод исходного данного
    // вычисления по формулам:
```

```
    l=2.*PI*r;
    s=PI*r*r;
    v=4.‘PI*r*r*r/3.;
    // Вывод результатов
    printf("r=%6.2f, l=%6.2f, s=%6.2f, v=%6.2f\n ", r, I, s, v);
} // End of main
```

Многие ошибки начинающих программистов вызваны незнанием информации о типах данных. Объявление переменной в программе указывает ее имя и тип. Тип переменной определяет способ хранения переменной, а именно сколько памяти выделено для переменной и как ее значение размещено в этом объеме. Тип также определяет диапазон значений, которые может принять переменная, и операции, которые можно к ней применить. Способ хранения переменных вещественного типа (double, float) радикально отличается от способа хранения данных целых типов (int, char). Тип констант, если они есть в тексте программы, определяется их записью. Так константа 2 будет целочисленной, а константы 2.0 или 2. будут вещественными. При вычислении значений выражений, в которых смешаны данные разных типов, выполняется преобразование (приведение) типов. Например, если в программе объявлены целочисленные переменные:

```
int a=1, b=2;
float f;
```

и выполнено деление, а значение присвоено переменной вещественного типа:

```
f=a/b;
```

то результат будет равен 0.0, потому что сначала выполняется целочисленное деление, а потом присваивание. Чтобы результат был равен 0.5, нужна запись с использованием явного преобразования типов:

```
f=(float)a/float(b);
```

Особенности, связанные с преобразованием типов, рассмотрим на следующем примере. Дано значение угла, измеренное в градусах, минутах и секундах. Требуется перевести значение угла в радианную меру. Формула вычисления:

$$\lambda' = \frac{\lambda \cdot 360}{2 \cdot \pi}.$$

```
// программа имеет скрытую ошибку
#include <stdio.h>
#include <math.h>
void main(void)
```

```
{
        int Grad, Min, Sec; // исходные данные всегда целые
        float GRAD;// новая переменная для
                                // вычисления полного значения угла
        float Rad;              // итоговое значение
        printf(("Введите значение угла в градусах, минутах и секундах:\n");
        scanf("%d%d%d", &Grad, &Min, &Sec);
        // полное значение угла:
        GRAD=Grad+Min/60+Sec/3600;
        // перевод в радианы:
        Rad=Grad*M_PI/360; // константа M_P1 объявлена в <math.h>.
        printf (("Значение угла в радианах: %12.6f\n",Rad);
} // End of main
```

Для проверки стоит выполнить два тестовых примера, вот их результаты:

```
Введите значение угла в градусах, минутах и секундах:
2 0 0
Значение угла в радианах: 0.017453
Введите значение угла в градусах, минутах и секундах:
1 59 59
Значение угла в радианах: 0.008727
```

Как видим, почти одинаковые входные данные дают разные результаты. Источником ошибки величиной в один градус является строка, вычисляющая полное значение угла. При делении чисел целого типа результат равен нулю, поэтому значения минут и тем более секунд отброшены при вычислении GRAD.

Для устранения ошибки используется явное преобразование типов.

```
#include <stdio.h>
#include <math.h> void main(void)
{
        int Grad, Min, Sec;
        float Rad;
        float GRAD;
        printf("Введите значение угла в градусах, минутах и секундах\n");
        scanf("%d%d%d", &Grad, &Min, &Sec);
        // полное значение угла:
```

```
    GRAD=(float)Grad+(float)Min/60.+(float)Sec/3600.;
    // перевод в радианы
    Rad=GRAD*M_PI/360;
    printf("Значение угла в радианах=%12.6f\n",Rad);
} // End of main
```

Рассмотрим особенности вычислений с использованием данных целого типа. Сформулируем содержательную задачу. Пусть для покупки подарков выделена некоторая сумма в рублях. Требуется узнать, сколько можно купить подарков, если известна цена одного подарка, и определить, сколько денег останется. Наоборот, если нужно купить N подарков, то найти наибольшую стоимость одного подарка и количество оставшихся денег.

Приведем решение задачи в целых числах.

```
#include <stdio.h>
void main(void)
{
    int Sum;    // выделена сумма денег
    int Cost;   // стоит один подарок
    int Сои;    // количество, которое можно купить на эту сумму
    int N; // количество, которое хочется купить
    int Rest;   // останется денег
    printf("Введите сумму и цену одного подарка\n");
    scanf("%d%d", &Sum, &Cost);
    // найдем количество и остаток:
    Cou=Sum/Cost; // целочисленное деление
    Rest=Sum%Cost; // остаток от деления
    printf("Можно купить %d подарков, останется %d рублей.
                                                \n",Сои,Rest);
    // найдем стоимость и остаток
    printff'Сколько подарков хочется купить?\n");
    scanf("%d", &N);
    Cost=Sum/N;
    Rest=Sum % N);
    printf("Наибольшая стоимость подарка %d\n",Cost);
    printf("Останется %d рублей.\n", Rest);
} // End of main
```

Операции целочисленного деления и вычисления остатка от деления можно применять только к данным целых типов. Если такие операции нужно выполнить для данных вещественных типов, используются функции ceil, floor, fmod библиотеки <math.h>. Функция ceil округляет вещественное число вверх, функция floor округляет вниз, а функция fmod находит остаток от деления по модулю. Все функции имеют аргументы типа double и возвращают значение типа double. Приведение типа при переходе от float к double необязательно, так как не приведет к потере данных.

Рассмотрим тот же пример с использованием вещественного типа данных. Все денежные суммы должны вычисляться в рублях и копейках, следовательно, переменные Sum, Cost и Rest имеют вещественный тип. При вычислениях необходимо округление вниз для получения целочисленного значения и округление с точностью до копеек. Деление должно быть по модулю.

Теперь покажем решение этой же задачи в вещественных числах.

```
#include <stdio.h>
#include <math.h>
void main(void)
{
	float Sum; // выделена сумма денег
	float Cost; // стоит один подарок
	int Cou;	// количество, которое можно купить на эту сумму
	int N; // количество, которое хочется купить
	float Rest; // останется денег
	printf("Введите сумму и цену одного подарка\n");
	scanf("%f%f", &Sum, &Cost);
	// найдем количество и остаток
	Cou=(int)floor(Sum/Cost);
	// целое число получено округлением вниз
	Rest=fmod(Sum, Cost); // остаток от деления.
	Printf("Купили %d подарков, останется %6.2f рублей .\n", Сои, Rest);
	// найдем стоимость и остаток
	printf("Введите число подарков, которое хочется купить\п");
	scanf("%d", &N);
	Cost=Sum/(float)N;
	// точность вычислений должна быть 0,01
	Cost=floor(Cost* 100)/100.;
	Rest=fmod(Sum, Cost);
```

```
    printf("Наибольшая стоимость подарка %6.2f руб.\n", Cost);
    printf("Останется %6.2f руб.\п", Rest);
} // End of main
```

Если этот пример получить как точную копию первого, то хорошо будет работать только первая формула вычислений. Вторая даст ошибку, вызванную наличием погрешности вычислений, которая должна быть менее одной копейки. Для устранения погрешности добавлена строка

```
Cost=floor(Cost*100)/100.;
```

1.2. Реализация линейной структуры алгоритма

Любой язык программирования включает в себя средства описания данных и средства описания действий.

Средства описания данных декларируют элементы и тины данных, используемые в программе.

Средства описания действий указывают действия, исполняемые программой.

Простейшим средством описания действий является выражение. *Выражение* задает действия, которые вычисляют единственное значение. Состоит из констант, переменных, а также знаков операций и скобок. Элементы данных, к которым применяется операция, называют операндами.

Методы для описания синтаксиса выражений автоматически распространяются на остальные средства ЯП.

ЯП вносят некоторую специфику в запись математических выражений. Например, математическое выражение

$$\frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

на ЯП записывается в виде

```
(–b + sqrt(b * b – 4 * a * c))/(2 * a)
```

Обычно в ЯП используется смесь нотаций (способов записи выражений)-, большинство операций записывается между операндами, однако некоторые операции записываются перед их операндами, другие – после их операндов. В примере операции +, * и / записаны между их операндами, однако минус записан перед b (в сегменте -b), а sqrt записана перед выражением, к которому она применяется. Различают три нотации: инфиксную, префиксную и постфиксную. В качестве примеров будем рассматривать бинарные операции. Бинарной называют операцию, применяемую к двум операндам.

В инфиксной нотации бинарная операция записывается между ее операндами: a+b. В префиксной нотации операция записывается первой: + a b, а в постфиксной нотации операция записывается последней: a b +.

Выражение может заключаться в скобки, это не влияет на его значение. Выражение E. как правило, имеет то же значение, что и (E).

Префиксную и постфиксную нотации называют бесскобочными. Причина – для определения операндов скобки не нужны.

Выражение в префиксной нотации определяется так:

префиксная нотация константы или переменной это сама константа или переменная;

применение операции op к подвыражениям E_1 и E_2 записывается в виде op E_1 E_2. Преимущество: префиксное выражение декодируется простым просмотром слева направо.

В такой нотации нет никакой неоднозначности и нет необходимости применять скобки для точного определения последовательности операций при вычислении выражения. Поскольку свободный от скобок способ записи изобрел польский математик Лукашевич, то для обозначения этой нотации применяется название польская запись.

Сумме x и y соответствует префиксная запись + x y.

Произведение + x y и z задается префиксной записью * + x y z.

Значение префиксного выражения + 20 30 равно 50.

Выражение * + 20 30 60 «сворачивается» до вида * 50 60, значение которого равно 3000.

Выражение * 20 + 30 60 равно * 20 90, которое, в свою очередь, равно 1800.

Префиксная нотация может применяться только к операциям с фиксированным количеством k $(k > 0)$ операндов.

Количество операндов операции определяет ее *арность* (*местность*).

Применение операции op с арностью k $(k > 0)$ к выражениям E_1, E_2, ..., E_k записывается как

op E_1 E_2 ... E_k

Здесь i-е выражение справа от op является ее i-м операндом (для $1 < i < k$). Специальные варианты префиксной нотации позволяют записывать операции с переменным количеством операндов:

read (a, b, c) (max x y)

Последний вариант называют кембриджской польской записью. В кембриджской польской записи операция и ее аргументы заключаются в скобки. Эта запись

используется в языке LISP. В польской кембриджской записи обычное выражение $(a + b) \cdot (c - a)$ будет выглядеть следующим образом: (* (+ a b)(-c a)).

Для «ручного» преобразования в префиксную нотацию в исходном выражении ищется последняя (по выполнению) операция. Эта операция группирует (в префиксной нотации) свои операнды, затем выбирается предпоследняя операция и группирует свои операнды и т. д.

Представим шаги преобразования выражения в префиксную форму. Шаги будем отделять друг от друга стрелкой следования $\rightarrow$. Для ограничения области действия операций будем использовать вспомогательные скобки группировки:

a + b + c → + a (b * c) → + a * b c

a * b + c → + (a * b) c → + * a b c

С помощью префиксной нотации можно вычислить любое выражение за одни просмотр. Однако надо знать заранее количество операндов у каждой операции. Как следствие, бинарное вычитание и унарный минус приходится обозначать различными символами. Впрочем, можно применить кембриджскую польскую запись со скобками.

Кроме экономии на отсутствии скобок в выражениях, префиксная нотация дает следующие дополнительные преимущества:

Префиксная нотация удобна для представления операций с любым количеством операндов, в этом смысле она считается наиболее общей формой. Для записи любого выражения достаточно знать лишь одно синтаксическое правило. Например, чтобы создать любое выражение на функциональном языке, нужно освоить только кембриджскую польскую запись, и после этого вам будут понятны правила синтаксиса языка.

Префиксное выражение достаточно легко вычислить механически, поэтому перевод префиксных выражений в простую последовательность кодов очень прост.

Простоту перевода в последовательность кодов проиллюстрируем следующим алгоритмом. Префиксное выражение PE можно вычислить с помощью стека:

1. Если очередной элемент в выражении PE обозначает символ операции, то заносим его в вершину стека. Заносим в счетчик операндов количество операндов, необходимых для данной операции.

2. Если следующий элемент является операндом, помещаем его в вершину стека.

3. Если верхние n элементов в стеке содержат набор операндов, достаточный для вычисления верхней n-арной операции, то эта операция применяется к операндам. Например, если символ умножения был последней операцией, записанной в стек, и в него же добавили два операнда. После выполнения операции ее символ и операнды из стека удаляются. Вместо них заносится результат.

Правда, здесь есть некоторая сложность – после занесения очередного операнда в стек приходится проверять, имеется ли достаточное количество операндов для

применения соответствующей операции. Для отказа от такой проверки следует воспользоваться постфиксной нотацией.

Выражение в постфиксной нотации определяется так:

постфиксная нотация константы или переменной – это сама константа или переменная;

применение операции op к подвыражениям E_1 и E_2 записывается в виде E_1 E_2 op.

Постфиксную нотацию иначе называют обратной польской записью.

Сумма x и y соответствует постфиксной записи x y +.

Произведение x y + и z записывается в следующей форме: x y + z *.

Значение выражения 20 30 + равно 50.

Значение выражения 20 30 + 60 * равно 50 60 * или равно 3000.

Значение выражения 20 30 60 + * равно 20 90 * или равно 1800.

Преимущество: постфиксные выражения могут вычисляться механически, с помощью структуры данных типа стек. Так как в постфиксной нотации операция записывается сразу за своими операндами, то при считывании символа операции се операнды уже известны. Следовательно, вычисление постфиксного выражения ЕР выполняется по алгоритму:

1. Если следующий элемент является операндом, помещаем его в стек.

2. Если очередной элемент обозначает символ n-арной операции, то п ее операндов должны находиться в верхних ячейках стека. Заменяем эти операнды на результат выполнения соответствующей операции.

Словом, алгоритм вычислений прост и легко реализуем. В большинстве компиляторов этот алгоритм является основой для генерации кода, вычисляющего выражения. В ходе компиляции выражения часто переводятся в постфиксную нотацию, а для вычисления выражения генератор кода применяет описанный выше алгоритм.

Для «ручного» преобразования исходного выражения в постфиксную нотацию в нем ищется первая (по выполнению) операция. Эта операция группирует (в постфиксной нотации) свои операнды, затем выбирается вторая операция и группирует свои операнды и т. д.

a + b + c → (a + b) c * → a b + c *

a * b + c → (a b *) + c → a b * c +

Формально подобное преобразование можно выполнять но алгоритму Дейкстры:

1. Проанализировать очередной элемент па входе.

2. Если это операнд, передать его на выход.

3. Если это «(», поместить ее в стек.

4. Если это операция, то:

если в вершине стека «(», поместить операцию в стек;

если у входной операции приоритет выше, чем у операции в вершине стека, поместить в стек новую операцию;

в противном случае – извлечь операцию из стека и отправить се на выход, а затем повторить шаг 4.

5. Если это «)», извлекать из стека операции на выход до тех пор, пока в вершине стека не окажется «(». Извлечь «(» из стека и уничтожить.

6. Если на входе еще есть элементы, перейти к шагу 1.

7. Если на входе нет элементов, выталкивать из стека все операции на выход.

В инфиксной нотации операции проставляются между их операндами. В сумме a и b операция сложения записывается между a и b.

Преимущество: легко читается человеком. Правда, хотя инфиксная нотация и широко распространена, ее использование создает определенные проблемы:

1. Поскольку инфиксная нотация определена лишь для бинарных операций, язык программирования не может использовать только одну эту форму записи для выражений, по обязательно должен сочетать ее с префиксной (постфиксной). Такое смешение делает, соответственно, компиляцию более сложной. Унарные операции и вызовы функций с несколькими аргументами не могут записываться в инфиксной нотации.

2. При появлении в выражении нескольких инфиксных операций для устранения неоднозначности приходится использовать скобки.

Действительно, как декодировать выражение $a + b \cdot c$? Это сумма a и $b \cdot c$? Или это произведение $a + b$ и c?

Для ответа на эти и другие вопросы вводятся понятия приоритетности и ассоциативности операций.

Приоритетность определяет порядок применения операции в выражении. Операция высокого приоритета выбирает свои операнды перед операцией низкого приоритета.

Рассмотрим выражение: $a + b \cdot c$.

В нашем выражении операция умножения выбирает свои операнды (и выполняется) перед операцией сложения.

Обычно операции с одинаковым приоритетом группируются слева направо:

$$4 - 2 + 1 = (4 - 2) + 1.$$

Точнее, так группируются левоассоциативные операции $(+, -, \cdot, /)$.

Ассоциативность задаст порядок группировки в выражении операций с одинаковым приоритетом.

Левоассоциативная операция – подвыражения, содержащие ее множественные вхождения, группируются слева направо:

$$b \cdot b - 4 \cdot a \cdot c = (b \cdot b) - \big((4 \cdot a) \cdot c\big).$$

Правоассоциативная операция – подвыражения, содержащие ее множественные вхождения, группируются справа налево:

Как видим, правоассоциативной является операция возведения в степень.

Операции, определяемые комбинацией символов, нельзя классифицировать в терминах префиксной, инфиксной и постфиксной нотации. Например, в выражении

```
if a > b then a else b
```

совместно используются ключевые (служебные) слова if, then. else.

Значащими компонентами этого выражения являются условие a > b и выражения a и b.

Если значением условия является true, то значением выражения является a, в противном случае – b.

Говорят, что операция имеет смешанную нотацию, если ее значащие компоненты перемежаются со служебными словами (или символами).

Каждая из рассмотренных нотаций для выражений имеет свои достоинства и недостатки.

Инфиксная нотация обеспечивает довольно естественное представление для большинства арифметических, логических выражений и выражений сравнения. Правда, приходится использовать сложные неявные правила приоритетности и ассоциативности операций (они меняются от языка к языку), а также скобки. Кроме того, не двухместные операции все равно записываются в префиксной (постфиксной) нотации. Все эти проблемы существенно усложняет компиляцию инфиксных выражений.

Конечно, в инфиксной нотации необходимость учета приоритетности и ассоциативности операций можно свести к минимуму за счет скобок, но тогда понадобится огромное количество скобок. Проблема скобок также присутствует в кембриджской польской и в обычной математической префиксной записи.

В польской нотации вообще не применяются скобки, но нужно заранее знать количество операндов у каждой операции, что бывает трудно выполнить. Например, если речь идет об операциях, которые создаются программистами.

Префиксная и постфиксная нотации легко справляются с записью операций, имеющих различные количества операндов.

Наконец, человеку трудно читать сложные выражения, записанные в польской нотации.

Как правило, вычисленное значение выражения нужно где-то запоминать. Такую задачу решает операция присваивания. Присваивание – очень важное и популярное

действие в императивных языках программирования. Вместе с тем присваивание является не совсем обычной операцией. Рассмотрим ее специфику, предварительно договорившись о смене терминологии.

Будем теперь называть операнды выражения объектами данных. Это позволит учесть особенности их обработки в памяти компьютера. Упрощая, можно сказать: объект данных – это операнд, находящийся в ячейке памяти. Точнее, в ячейке находится его значение.

Присваивание – это элементарная операция, изменяющая связывание объекта данных со значением. Как ни странно, такое изменение считается побочным эффектом присваивания.

Поскольку присваивание является двухместной операцией, применительно к ее инфиксной нотации будем различать селевый и правый операнды (аргументы).

В языке Pascal присваивание имеет такой смысл: скопировать в один (левый) объект данных значение, содержащееся в другом (правом) объекте данных такого же тина, но не возвращать явный результат. При этом изменение значения левого объекта считается неявным результатом (побочным эффектом) присваивания.

Несколько забегая вперед, условимся описывать двухместные операции в следующем формате:

символ операции: тип первого аргумента × тип второго аргумента → тип результата

Типом называется существенная характеристика объекта данных. Она задает допустимые значения, операции и способ размещения объекта в памяти.

Соответственно, спецификация операции присваивания в языке Pascal примет вид:

присваивание (:=) : тип × тип → пустой тип (void)

В языке С смысл у присваивания несколько иной: скопировать в левый объект данных значение, содержащееся в правом объекте данных такого же типа, а также вернуть результат – объект данных, содержащий копию присвоенного значения. Спецификация операции присваивания записывается в форме:

присваивание (=) : тип × тип → тип

Рассмотрим такое присваивание: a=a. Здесь интересно проследить различные интерпретации ссылок па переменную a. Стоящее справа a ссылается на величину, содержащуюся в переменной с этим именем. Подобные ссылки называются правосторонними значениями (операции присваивания) пли *r*-значениями объекта данных. В свою очередь, стоящее слева a ссылается на местоположение объекта, в котором будет содержаться новое значение (ссылка на местоположение). Такие ссылки называются левосторонними значениями (операции присваивания), или *l*-значениями. Используя эти термины, операцию присваивания нужно определить следующим образом:

1. Вычислить *l*-значение первого операнда выражения.
2. Вычислить *r*-значение второго операнда выражения.

3. Присвоить вычисленное *l*-значение вычисленному *l*-значению объекта данных;

4. Возвратить вычисленное *l*-значение как результат выполнения операции.

Таким образом, операция присваивания в языке С возвращает присвоенное *l*-значение объекта данных. В языке С широко используются обе возможности операции присваивания: и как механизма изменения значения объекта данных (через его *l*-значение), и как функции, возвращающей значение (через *r*-значснис объекта данных).

В общем случае операндами выражения могут быть переменные, константы, функции и подвыражения в скобках.

Вычисление значений переменных происходит путем выборки их из памяти. Константы пли вычисляются тем же способом, пли же их значения уже присутствуют в выражении и не требуют выборки из памяти. Вычисление функции сводится к исполнению операторов се тела. Вычисление подвыражения в скобках требует, в свою очередь, вычисления содержащихся в нем операндов.

Если ни один из операндов в операторе не имеет побочных эффектов, то порядок вычисления операндов значения не имеет и компилятор может установить его сам. решая задачу оптимизации вычислений. Следовательно, особый интерес представляет именно наличие у операндов побочных эффектов.

Практически наиболее важен побочный эффект функции. Он возникает, когда функция изменяет один из своих параметров или глобальную переменную. Глобальная переменная объявляется вне функции, но доступна в ней.

Рассмотрим выражение:

n × func(m) + n

Прежде чем выполнять умножение, следует извлечь из памяти *r*-значение переменной n, *r*-значснис переменной m и вычислить func(m). Для сложения нужен результат операции умножения и значение п. Чтобы сократить время вычислений, целесообразно извлечь значение л из памяти только один раз, а затем в ходе вычислений использовать его два раза. Кроме того, вычисление func(m) и извлечение n из памяти не должны зависеть друг от друга. Теперь предположим, при вычислении функции из-за побочного эффекта изменяется значение n. В этой ситуации точный порядок вычисления приобретает первостепенное значение. Например, если вначале в ячейке памяти для n хранилась 1, а func(m) возвращает результат 4 и также изменяет величину n на 1. тогда различные порядки вычислений приводят к разным результатам:

вычисления в порядке слева направо дают результат: 1 × 4 + 2 = 6.

n вычисляется только один раз, причем до вычисления func(m): 1 × 4 + 1 = 5.

значение func(m) вычисляется до вычисления n: 4 × 2 + 2 = 10.

Во всех этих вариантах вычисления корректны (сточки зрения правил языка), но отличаются порядком вычисления операндов.

Заметим, что в математике функции не имеют побочных эффектов, поскольку здесь нет понятия переменной. То же самое справедливо и для языков функционального программирования. В математике и языках функционального программирования функции гораздо проще для понимания, чем в императивных языках, потому что их контекст никак не связан с их смыслом.

Проблема порядка вычисления операндов и побочных эффектов имеет два возможных решения:

не допускать использования в выражениях функций с побочным эффектом;

разрешать побочные эффекты, по в определении языка задать точный порядок вычисления операндов в выражениях.

Запретить побочный эффект функций довольно трудно, поскольку это существенно снижает гибкость программирования. Ведь речь идет об исключении целого арсенала функций с двусторонними параметрами и запрете на доступ к глобальным переменным. Отказ от функций, возвращающих несколько значении, резко сужает возможности программистов, а обращение к глобальным данным, вне механизма передачи параметров, считается важным методом увеличения эффективности и скорости выполнения программ. В частности, компиляторы весьма широко применяют глобальный доступ к таблицам символов.

С другой стороны, установка строгого порядка вычислений конфликтует с методами оптимизации программ. В рамках данного подхода из определения языка должен следовать совершенно ясный порядок вычисления операндов, чтобы программист мог правильно использовать побочные эффекты. Однако многие оптимизирующие компиляторы изменяют порядок вычислений операндов внутри выражений. Это означает: при строго заданном порядке вычисления определенные виды оптимизации становятся невозможными. Таким образом, лучшего решения просто нет, что и подтверждается существующими языками.

Для операторов возможность побочных эффектов обычно допускается. Самый распространенный оператор присваивания всегда производит побочный эффект, изменяя значение элемента данных. Достаточно часто побочный эффект одного оператора влияет на исходные данные другого оператора.

Особым видом побочного эффекта считают аварийное (неуспешное) завершение функции, приводящее к генерации состояния ошибки. Такой побочный эффект характерен не только для высокоуровневых функций, созданных программистом, но и для элементарных операций (примеры: переполнение, деление на ноль и т. д.). Запрещать, подавлять подобные эффекты, конечно, нс принято. Нужно лишь продумывать всевозможные условия их появления и варианты влияния на ход вычислений.

Очень часто для организации ветвления применяют сложные условия, состоящие из набора простых выражений отношения. В роли «клея», сочленяющей)

выражения отношения, в них выступают логические (булевы) операции И (&& в языке С) и ИЛИ (| | в языке С). В языке С выполнение этих операций соответствует короткой схеме вычислений. По этой схеме второй операнд логической операции вычисляется не всегда. В частности, справедливо следующее:

значением булева выражения a | | b будет истина, если значение a – истина;

значением булева выражения a && b будет ложь, если значение a – ложь.

В данных случаях нет нужды вычислять значение операнда b.

Приведем два примера, записанных на языке С:

```
(A == 0) || (B/A > C)
```

и

```
(I <= MAX) && (Z[I] > N)
```

Смысл этих выражений состоит в следующем: вычисление правого операнда булевой операции может привести к побочному эффекту – возникновению ошибки, а именно:

деление на ноль – для верхнего примера:

выход индекса за пределы допустимого диапазона – для нижнего примера.

Левый операнд призван предотвратить такую ошибку.

Правый операнд не вычисляется:

если левый операнд (A == 0) получил значение истина,

если левый операнд (I <= MAX) получил значение ложь.

Увы, но на уровне реальной реализации большинства языков эта идея не срабатывает. поскольку оба операнда вычисляются еще до сканирования символа булевой операции. Многие ошибки возникают из-за неоправданного расчета па то. что значение левого операнда булевой операции может отменить вычисление ее правого операнда. Самая надежная защита от подобных ошибок предложена в языке Ada, где в дополнение к обычным булевым операциям and и or. которые не вычисляются но короткой схеме, введены две специальные булевы операции, and then и or else, явным образом осуществляющие вычисление по короткой схеме. Например, на языке Ada выражение

```
(A = 0) or else (B/A > C)
```

не может сгенерировать ошибку деления на ноль, так как при A = 0 вычисление всего выражения прекращается и его значение считается истинным.

СПИСОК ЛИТЕРАТУРЫ

1. Игошин В.И. Математическая логика и теория алгоритмов : учеб, пособие для студ. высш. учеб, заведений / В. И. Игошин. – 2-е изд. стер. – М.: Издательский центр «Академия». 2008. – 448 с.

2. Глухов М. М., Шишков А. Б. Математическая логика. Дискретные функции. Теория алгоритмов: Учебное пособие. – СПб.: Издательство «Лань», 2012. – 416 с.

3. Геут К. Л. Математическая логика и теория алгоритмов : учсб.-метод. пособие / К. Л. Геут, С. С. Титов. – Екатеринбург : УрГУ ПС, 2017. – 85 с.

4. Мендельсон Э. Введение в математическую логику. – М.: Наука, 1976. – 320 с.

5. Зюзьков В. М. Математическая логика и теория алгоритмов : учебное пособие / В. М. Зюзьков. – Томск : Эль Контент, 2015. – 236 с.

6. Успенский В. Л. Труды но нематематике / В. А. Успенский. 2-е изд,, испр. и доп. : В 5 кн.– М. : ОГИ : Фоад «Магматические этюды», 2012. – Кн. 2 : Философия (с приложением трёх писем П. .П. Трауберг к автору). – 2014. – 566 с.

7. Лихтарников Л. М., Сукачева Т. Г. Математическая логика. Курс лекций. Задачник-практикум и решения: Учебное пособие. 4-е изд., стер. – СПб.: Издательство «Лань», 2009. – 288 с.

8. Вялый М., Подольский В., Рубцов А., Шварц Д., Шень А. Лекции по дискретной математике. 448 с. https://publications.hse.ru/mirror/pubs/share/direct/393719078.pdf

9. Пальчунов, Д. Е. Математическая логика и теория алгоритмов : учеб, пособие / Д. Е. Пальчунов ; Новосиб. гос. ун-т. - Новосибирск, 2016. – Ч. 1. – 96 с.

10. Барашев В.П.. Унучек С.А. Дискретная математика : Учебное пособие Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Московский государственный технический университет радиотехники, электроники и автоматики». – М., 2012. –268 с., электронное издание.

11. Чашкин А. В. Дискретная математика : учебник для учреждений высш. проф. образования / А. В. Чашкин. – М.: Издательский центр «Академия», 2012. – 352 с.

12. Гринченков Д.В. Математическая логика и теория алгоритмов для программистов : учебное пособие / Д.В. Гринченков, С.И. Потоцкий. – М.: КНОРУС, 2010. – 208 с.

13. Вайнштейн К. В. Математическая логика и теория алгоритмов : учеб, пособие / Ю. В. Вайнштейн, Т. Г. Пенькова, В. И. Вайнштейн. – Красноярск : Сиб. федер. ун-т, 2019. – 110 с.

14. Новиков Ф. А. Дискретная математика для программистов. Учебник для вузов. 3-е изд. – СПб.: Питер, 2009. – 384 с.

15. Марченков С. С. Основы теории булевых функций. М.: ФИЗМАТЛИТ, 2014. – 136 с.

16. Гаврилов Г. П., Сапоженко А. А. Задачи и упражнения по дискретной математике: Учеб, пособие. – 3-е изд., перераб. – М.: ФИЗМАТЛИТ, 2005. – 416 с.

17. Кормен Т. Х., Лензерсон, Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. 2-е издание : Пер. с англ. – М. : Издательский дом «Вильямс», 2011. – 1296 с.

18. Алексеев В.В. Теория алгоритмов. Учебно-методическое пособие. СарФТИ НИЯУ МИФИ, 2021. – 100 с.

19. https://education.yandex.ru/handbook/algorithms

20. Ахо А. В., Хопкрофт Д. Э., Ульман Д. Д. Структуры данных и алгоритмы : Пер. с англ. : М. : Издательский дом «Вильямс», 2003. – 384 с.

21. Граничин О.Н. Рандомизированные алгоритмы в задачах обработки данных и принятия решений. Системное программирование. Санкт-Петербургский государственный университет, Т. 6, № 1. С. 141–162.

22. Регеда В. В., Регеда О. Н. Основы алгоритмизации. Методические указания по курсу «Информатика». Пенза: Издательство Пензенского государственного университета, 2008. – 56 с.

23. Туаева М. Т. Алгоритмы. Основы алгоритмизации: Учебное пособие. – Владикавказ: Владикавказский торгово-экономический техникум, 2020. – 51 с.

24. Алгоритмы. Основные алгоритмические конструкции [Текст]: сб. задач / сост. С.А. Рогозин. – Челябинск: Изд-во Челяб. гос. пед. ун-та, 2008. 42 с.

25. Глухова Л.А. Основы алгоритмизации и программирования : лаб. практикум для студ. спец. I-40 01 01 «Программное обеспечение информационных технологий» дневн. формы обуч. В 4 ч. Ч. 3 / Л.А. Глухова, Е П. Фадеева, Е.Е. Фадеева. – Минск : БГУИР, 2007. – 52 с.

26. Макконнелл Дж. Основы современных алгоритмов. 2-е дополненное издание. Москва: Техносфера, 2004. – 368с.

27. Орлов С. А. Теория и практика языков программирования: Учебник для вузов. Стандарт 3-го поколения. – СПб.: Питер. 2013. – 688 с.

28. Анисимов А.Е. Ведение в алгоритмизацию задач / А.Е. Анисимов. – Ижевск: Изд-во «Удмуртский университет», 2017. – 44 с.

29. Конова Е. А. Алгоритмы и программы. Язык С++ : учебное пособие для СПО / Е. А. Конова, Г. А. Поллак. – 3-е изд., стер. – Санкт-Петербург : Лань, 2022. – 384 с.

30. Паклина В.М., Паклина Е.М. Основы алгоритмизации и программирования. – Екатеринбург: ГОУ ВПО УГТУ−УПИ, 2009. – 91 с.

31. Ворожцов А.В., Винокуров Н.А. Лекции Алгоритмы: Построение, анализ и реализация на языке программирования Си. – Москва: МФТИ, 2007. – 452 с.

32. Мусатов Д.В. Сложность вычислений : конспект лекций. – Москва: МФТИ, 2017. – 224 с.

33. Седжвик Р. Фундаментальные алгоритмы на С++. Анализ / Структуры данных / Сортировка / Поиск : Пер. с англ. / Роберт Седжвик. – К.: Издательство «ДиаСофт», 2001. – 688 с.

34. Имран А. 40 алгоритмов, которые должен знать каждый программист на Python. – СПб.: Питер. 2023. – 368 с.

35. Hetland M. L. Python Algorithms: Mastering Basic Algorithms in the Python Language. – 2010. – 337 p.

36. Пинчук В.П. Анализ и построение алгоритмов. Краткий конспект лекций для спец. КСС. – Запорожье: ЗНТУ, 2012.

37. Филиповский В.М. Основы программирования и алгоритмизации. Ч. 2. Практикум по алгоритмизации: Учебное пособие. – СПб.: СПбПУ. 2022. – 71 с.

38. Кормен Т. Х. Алгоритмы: вводный курс. : Пер. с англ. – М. : ООО «И.Д. Вильямс», 2014. – 208 с.

39. Стивенс Р. Алгоритмы. Теория и практическое применение / Род Стивене. – Москва : Издательство «Э». 2016. – 544 с.